Two-Timescale Joint Optimization of Task Scheduling and Resource Scaling in Multi-Data Center System Based on Multi-Agent Deep Reinforcement Learning

Shuangwu Chen, *Member, IEEE*, Jiangming Li, Qifeng Yuan, Huasen He, *Member, IEEE*, Sen Li, and Jian Yang, *Senior Member, IEEE*

***Abstract*—As a new computing paradigm, multi-data center computing enables service providers to deploy their applications close to the users. However, due to the spatio-temporal changes in workloads, it is challenging to coordinate multiple distributed data centers to provide high-quality services while reducing service operation costs. To address this challenge, this article studies the joint optimization problem of task scheduling and resource scaling in multi-data center systems. Since the task scheduling and the resource scaling are usually performed in different timescales, we decompose the joint optimization problem into two sub-problems and propose a two-timescale optimization framework. The short-timescale task scheduling can promptly relieve the bursty arrivals of computing tasks, and the long-timescale resource scaling can adapt well to the long-term changes in workloads. To address the distributed optimization problem, we propose a two-timescale multi-agent deep reinforcement learning algorithm. In order to characterize the graph-structured states of connected data centers, we develop a directed graph convolutional network based global state representation model. The evaluation indicates that the proposed algorithm is able to reduce both the task makespan and the task timeout while maintaining a reasonable cost.**

***Index Terms*—Multi-data center system, task scheduling, resource scaling, deep reinforcement learning, graph convolution network.**

I. Introduction

DRIVEN by the globalization of internet applications, service providers tend to deploy their applications in multiple geographically dispersed data centers, bringing services closer to users to shorten the time for interaction [1]. As a new computing paradigm, multi-data center computing is able to improve resource utilization and reduce the operation cost [2] by sharing the computing resources among multiple data centers in a cooperative manner. According to IDC's prediction [3], over 55% of the world's top 2000 companies will adopt the multi-data center system to provide service by 2025. However, the spatio-temporal dynamic of workloads in different clouds will lead to load imbalance and even overload, thus increasing the processing time and causing resource waste [4]. It is challenging to coordinate multiple distributed data centers to provide high-quality services while reducing service operation costs [5].

To cope with the dynamic changes of task load in the service requests, much literature has studied task scheduling algorithms in the multi-data center system, which dispatch tasks from high-load data centers to low-load data centers to achieve workload balance [6], [7]. Most of these studies focus on centralized task scheduling, which requires a centralized controller to schedule tasks uniformly according to the states of the distributed data centers [8], [9], [10]. Since the task requests need to be uploaded to the central controller for scheduling, centralized algorithms will cause additional request-scheduling delays [4]. Moreover, collecting the states from multiple data centers will bring extra communication overhead. In this context, some researches have attempted to develop a distributed task scheduling algorithm, where each data center makes its own scheduling decisions independently [11], [12]. However, the independent decision making tends to schedule all the tasks to the lightly loaded data centers, which would be overwhelmed instead. To address this problem, collaborative task scheduling has been proposed recently via the global states sharing [13], [14]. None of these methods considers the cost of data transmission, which is one of the most expensive resources in the multi-data center system [15]. Although task scheduling can temporarily alleviate the sudden increase of task load, it would be out of work when the workload exceeds the computing capacity of the multi-data center system. Meanwhile, there will be a waste of resources when the system is under low load. The resources are expected to be adjusted dynamically to match the long-term changes of workloads.

By enabling resource virtualization technology, service providers can flexibly scale up or down the resource instances

Received 1 June 2023; revised 24 August 2024; accepted 15 September 2024. Date of publication 24 September 2024; date of current version 15 October 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 62101525 and Grant U23A20275, in part by the China Environment for Network Innovations (CENI) under Grant 2016-000052-73-01-000515, and in part by the Guangxi Key Laboratory of Digital Infrastructure under Grant GXDIOP2023001. Recommended for acceptance by D. Medhi. *(Corresponding author: Huasen He.)*

Shuangwu Chen, Jiangming Li, Qifeng Yuan, Huasen He, and Jian Yang are with the University of Science and Technology of China, Hefei, Anhui 230027, China (e-mail: hehuasen@ustc.edu.cn).

Sen Li is with the Guangxi Key Laboratory of Digital Infrastructure, Nanning, Guangxi 530201, China.

Digital Object Identifier 10.1109/TPDS.2024.3467212

1045-9219 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:09:31 UTC from IEEE Xplore. Restrictions apply.

of the data centers according to the dynamic workloads [16]. Traditional researches focus on reactive resource scaling, where predefined rules or thresholds are set to trigger the resource scaling [17], [18]. However, the reactive scaling may cause the hysteresis of decisions, which scales the resources according to the current or past task arrivals. To solve this issue, the proactive scaling methods have been proposed, which adjust the resource configuration according to the predicted workload [19], [20], [21]. Due to the long time consumption of loading the resource instance, resource scaling can hardly have a prompt response to the bursty arrivals of service requests. Generally, for ensuring the *Quality of Service* (QoS), we need to rent more computing resources to deal with the sudden increase of workload, which may cause unnecessary additional costs to the service provider [22].

In light of this situation, we jointly consider the task scheduling and the resource scaling in the multi-data center system, which to the best of our knowledge is studied by a paucity of literature. The task scheduling can promptly relieve the bursty arrivals of service requests, and the resource scaling can adapt well to the long-term changes of workloads. In order to improve economic benefits, service providers usually want to reduce operating costs, including data transmission costs and resource rental costs. Although renting less resource can reduce the cost, it may lead to more task timeout, thus reducing the service QoS, which is something service providers do not want to see. Therefore, we formulate a joint optimization problem with the goal of minimizing overall costs for data transmission and resource rental while avoiding task timeout. To solve this problem, we decompose the optimization problem into two sub-problems at two timescales. Considering the heterogeneity of computing resources of different data centers and the dynamic network delays between them, we conceive a *Directed Graph Convolutional Network* (DGCN) based global state representation model, which is used to coordinate the independent decision making of the data centers. Finally, we propose a novel *Two-Timescale multi-Agent Deep* reinforcement learning (2TAD) algorithm for both task scheduling and resource scaling. The main contributions of this paper are summarized as follows:

- We formulate a joint optimization problem of task scheduling and resource scaling to minimize the total costs used for data transmission and resource rental while avoiding task time out in the multi-data center system, which is rarely considered in the literature.
- Since the task scheduling and the resource scaling are usually performed in different timescales, we decompose the joint optimization problem into two sub-problems, and propose a two-timescale optimization framework. The short-timescale task scheduling can promptly relieve the bursty arrivals of computing tasks, and the long-timescale resource scaling can adapt well to the long-term changes of workloads.
- We propose a two-timescale multi-agent deep reinforcement learning algorithm for both task scheduling and resource scaling. In order to reduce the communication overhead used for state sharing, we adopt a DGCN based global state representation model to characterize the spatial dynamic of workloads in the multi-data center system.
- We conduct a performance evaluation using real multi-data center system environment parameters. Experimental results show that the proposed algorithm can effectively improve the quality of service by reducing the makespan and the timeout ratio of tasks while maintaining a reasonable cost.

The rest of this paper is organized as follows. Section II gives a comprehensive review of related work, and Section III presents the system models and the problem formulation. A two-timescale joint optimization framework is further proposed for decoupling this problem in Section IV. Then, we develop a multi-agent deep reinforcement learning based optimization algorithm for task scheduling and resource scaling in the multi-data center system in Section V. Evaluations are conducted in Section VI to evaluate the performance. Finally, a conclusive discussion is offered in Section VII.

II. Related Work

We have a review of existing researches on task scheduling and resource scaling in the multi-data center system, which are summarized as follows.

A. Task Scheduling

In recent years, multi-data center computing has attracted more and more service providers to deploy services in the data center by virtue of its flexibility and reliability. Since the computing resources rented by service providers are limited, how to schedule tasks among multi-data centers when the task load changes dynamically becomes a key issue. Yi et al. [6] used a *Deep Reinforcement Learning* (DRL) based approach for task scheduling among computing servers in a single data center to reduce the energy consumption. Marahatta et al. [7] designed a dynamic task scheduling scheme for virtualized data centers, which used bayesian classifiers and *Historical Scheduling Records* (HSR) to classify tasks, and adopted a heuristic algorithm to map tasks to corresponding VMs. These methods only considered a single data center environment, and could not be applied to a heterogeneous multi-data center system where the dynamic data transmission delay and cost between data centers might affect the task scheduling. To solve this problem, Zhu et al. [9] proposed a scheduling method based on *Matching and Multi-round Assignment* (MMA), which minimized the task makespan and the computing cost while satisfying reliability constraints. Tang [10] proposed a *Fault-tolerant Cost-effective Workflow Scheduling* algorithm (FCWS), which implemented workflow application scheduling to satisfy workflow reliability constraints. Chen et al. [24] proposed an *Online Workflow Scheduling algorithm based on Adaptive resource Allocation and Consolidation* (OWS-A2C), aiming to improve resource utilization that meets deadline constraints in multi-cloud environments and reduce the execution cost of scientific workflows. Kanbar and Faraj [25] proposed a *Region Aware DynamIc ScHeduling* (RADISH) model that can perform dynamic task scheduling and load balancing in the context of IoT multi-cloud

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:09:31 UTC from IEEE Xplore. Restrictions apply.

environments, reducing task scheduling delays. These works adopted a centralized control architecture to schedule tasks uniformly, which might cause additional communication overhead for collecting the states from multiple data centers. Meanwhile, the request-scheduling delay of the centralized controller in the forwarding decision process would significantly increase the task makespan [4]. To overcome these drawbacks, recent researches implemented a distributed task scheduling algorithm, in which each data center scheduled the tasks independently. Long et al. [11] proposed a distributed task scheduling algorithm based on *Limit Searching Algorithm* (LSA) and *Greedy Energy-aware Algorithm* (GEA), which performed the task assignment in a distributed, non-cooperative and competitive environment. Considering the resource competition among data centers, Zhang et al. [12] designed a distributed task scheduling algorithm based on multi-agent reinforcement learning, which attempted to optimize system delay. In the above works, each data center made its scheduling decision independently, which would make all the tasks to be scheduled to the lightly loaded data centers. Consequently, cooperative task scheduling among multiple data centers should be considered. Suzuki et al. [13] proposed a task scheduling method based on *Cooperative Multi-Agent Deep Reinforcement Learning* (Coop-MADRL), which improved the efficiency of task scheduling. To make a step forward, Ke et al. [14] used a *Multi-agent Computation Offloading scheme based on Deep Reinforcement Learning* (MCODRL) to minimize the weighted average cost, where a *Gated Recurrent Unit* (GRU) based global information sharing model was adopted for cooperation. However, none of these task scheduling algorithms takes into account the data transmission cost, which will increase the operation cost of the service providers. Most importantly, task scheduling can only alleviate the dynamic change of workload, but cannot solve the problem of task timeout caused by insufficient computing resources and the problem of resource waste caused by excessive resources.

B. Resource Scaling

Resource virtualization enables service providers to dynamically adjust the number of resource instances for matching workload. How to adaptively scale resources according to changes of workload is an essential problem, which has attracted many research attention. Traditional researches focused on threshold-based or rule-based reactive resources scaling [17], [18], [23]. Srirama et al. [17] designed a threshold-based scaling algorithm and used a dynamic binning strategy to deploy containers. Gambi et al. [18] proposed an adaptive cloud controller, which allocated resources to applications based on the Kriging model in order to meet the service quality requirements. To make a step forward, Kwan et al. [23] devised a threshold-based hybrid scaling technique named HYscale, which considered both vertical scaling and horizontal scaling. The above scaling methods required service providers to set static rules or thresholds, which might lead to poor scaling decisions due to the heterogeneous resource demand of different services [26]. To solve this problem, some studies employed learning-based algorithms to achieve adaptive scaling decisions. Rossi et al. [26] proposed a resource scaling strategy based on dynamic thresholds, which used *Multi-Agent Deep Reinforcement Learning* (MADRL) to dynamically update thresholds for each indicator. Laqbal et al. [27] adopted a polynomial regression prediction model to predict the makespan for the next interval, and then used a *Random Decision Forest* (RDF) to allocate the required resources. Xu et al. [19] proposed a GRU based prediction model to estimate the future workloads and used a reinforcement learning method to scale the resources. Yan et al. [21] used the attention based Bi-LSTM network to predict the changes of workload, and then a MADRL based elastic scaling method was proposed. Similarly, Kardani et al. [20] designed a resource scaling method based on DRL, which solved the resource shortage problem at the local VM level and the load balance problem at the system level. To cope with the unexpected events of the workload, in [28], Abdullah et al. proposed a novel burst-aware auto-scaling approach that combined workload prediction, resource prediction and scaling decisions to detect bursts of dynamic workloads. However, since the configuration and loading of resource instances usually take a long time, resource scaling cannot meet the sudden fluctuation of workload. In addition, renting more computing resources to ensure the QoS will increase the cost of the cloud service provider.

We summarize and compare the characteristics of the above studies and our method, as shown in Table I. From the above literature review, merely considering task scheduling or resource scaling is difficult to cope with dynamic changes of workload in the multi-data center system. Motivated by this fact, this paper proposes a two-timescale multi-agent deep reinforcement learning based joint optimization algorithm for both task scheduling and resource scaling.

III. System Model and Problem Formulation

In this section, we first give an overview of the multi-data center system, then introduce system models, and finally formulate a joint optimization problem. The major notations used in our models are presented in Table II.

A. System Overview

To provide better service for users in different regions of the world, service providers usually deploy their services in multiple data centers [29], which can greatly shorten the time of interaction. Considering task scheduling among multi-location data centers in a joint optimization framework can effectively reduce task queuing time and task timeout rate. By analyzing the load characteristics of geographically distributed data centers, the research in [30] found that there were differences in user behavior in different regions, resulting in different patterns of task requests in different data centers, with peaks and troughs complementing each other. As service providers often have strict budget controls to ensure their actual costs meet financial plans, they may only rent limited computing resources in each data center. If the workload was considered separately in each data center, when a burst of tasks arrive at a data center, the tasks have to queue up for a long time, which may result in task timeouts. When the task arrival rate of a data center is low, the data center

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:09:31 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON OF THE RELATED WORKS IN TASK SCHEDULING AND RESOURCE SCALING

References	Distributed Decision	Task Scheduling	Resource Scaling	Cooperation	Learning Based Algorithm	Solving Method
[6]	×	✓	×	×	✓	DQN
[7]	×	✓	×	×	×	Heuristic Algorithm
[9]	×	✓	×	×	×	MMA
[11]	✓	✓	×	×	×	LSA & GEA
[13]	✓	✓	×	✓	✓	MADRL
[14]	✓	✓	×	✓	✓	GRU & MARL
[18]	×	×	✓	×	×	Kriging Model
[23]	×	×	✓	×	×	Threshold-based
[19], [20]	×	×	✓	×	✓	DRL
[21]	✓	×	✓	✓	✓	MADRL
Proposed	✓	✓	✓	✓	✓	DGCN & MADRL

TABLE II
MAJOR NOTATIONS USED IN THE PAPER

	Symbol	Notation
System	$\mathcal{C}$, $\mathcal{G}$	the set of data centers, system graph
	$B_{c,c'}$, $d_{c,c'}$	WAN bandwidth, data transmission delay between data center c and c'
	$\delta_{c,t}$, $l_{c,\tau}$, $\beta_{c,\tau}$	computing resource of the data center c, CPU cycles required and data for $q_{c,\tau}$
Task Scheduling	$x_{c'}(q_{c,\tau})$	the task scheduling decide of $q_{c,\tau}$, here $x_{c'}(q_{c,\tau}) \in \{0,1\}$
	T^{mak}, T^{exe}, T^{que}, T^{com}	makespan, execution time, queue time, and transmission time of tasks
	$V^{com}(q_{c,\tau})$, $p^{com}_{c,c'}$	transmission cost of task $q_{c,\tau}$, the price of data transmission between c and c'
	Z	the maximum tolerable delay of the task
	$\boldsymbol{s}^S$, $\boldsymbol{a}^S$, R^S	local states, actions and rewards of task scheduling
Resource Scaling	I , ξ	resource instance, the computing resources of I
	$V^{ren}_{c,t}$, p^{ren}_{c}	resource rental cost, the price of I in data center c
	U	the maximum resource utilization
	$\boldsymbol{s}^L$, $\boldsymbol{a}^L$, R^L	local states, actions and rewards of resource scaling

will be idle, resulting in a waste of resources. If considering multi-location data centers together, we can schedule tasks from busy data centers to idle ones, thereby reducing the queuing time in busy data centers and avoiding resources waste in idle data centers. The task-schedulable multi-data center system has more flexible and powerful demand response capabilities, overcoming the limitations of demand response in a single data center. Hence, we consider task scheduling between multi-location data centers in a joint optimization framework instead of performing task processing separately in each data center.

We consider a typical multi-data center system consisting of multiple data centers spanning geographically dispersed locations, as shown in Fig. 1. Each data center contains a large volume of interconnected servers, which are usually virtualized as resource instances, such as *Virtual Machines* (VMs) and containers. Different data centers are connected together through WAN to form a computing network, which enables users to upload their computing tasks to the cloud services for further processing. In practice, the network bandwidth and latency between data centers vary widely. The spatio-temporal dynamic nature of task loads leads to unbalanced loads among data centers, which will reduce the QoS and increase the cost of cloud service providers. To address this problem, one viable solution is to scale the resource occupation elastically by increasing or releasing some resource instances according to their actual dynamic service burdens. Since the configuration and adjustment of data center resources usually take a long time (minutes level), it is difficult to have a prompt response

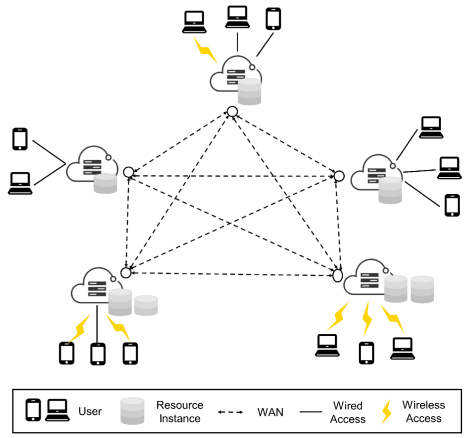


Fig. 1. An overview of multi-data center system.

to the bursty changes of computing tasks (milliseconds level), which may cause a large number of task time out. Alternatively, service providers can schedule tasks from busy data centers to idle ones, which is able to balance the workloads and take full use of the distributed computing resources among the data centers. However, scheduling tasks from a busy but close data center to an idle but distant one may increase the interaction delay. From a long-term perspective, we expect to increase the resource instances for the data centers which are chronically busy in order to improve the QoS. Whereas, we expect to reduce the resource instances for the idle ones, so as to save cost. Hence,

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:09:31 UTC from IEEE Xplore. Restrictions apply.

it is necessary to jointly optimize the task scheduling and the resource scaling in the multi-data center system.

B. Task Scheduling Model

We denote the set of data centers as $\mathcal{C} = \{1, \ldots, c, \ldots, C\}$. The resource scaling is performed periodically at regular time slots. Let t denote the time instant of resource scaling. Whereas, the tasks arrive randomly at the data center. Let τ denote the time instant of task arrival. As a large number of computing tasks may arrive within a time slot, the time interval of task scheduling is much less than that of resources scaling, i.e., $\Delta\tau \ll \Delta t$. Let $q_{c,\tau}$ denote the task that arrives at data center c in time instant τ. The execution of the task $q_{c,\tau}$ consumes a certain amount of computing resources denoted by $l_{c,\tau}$, which is usually quantified by the number of CPU cycles. Meanwhile, the offloading of task $q_{c,\tau}$ usually involves data transmission from the user device to the data center, which is quantified by the amount of data and is denoted by $\beta_{c,\tau}$. To maintain acceptable service quality, each task has a maximum tolerant time Z for computing the task in the data center.

Generally, the computing tasks are offloaded to the nearest data center for execution. Considering the maximum tolerable latency of tasks, if the nearest data center is heavily loaded, some tasks will be scheduled to other lightly loaded data centers immediately. Here, we define an indicator function of $x_{c'}(q_{c,\tau})$ as the scheduling decision:

$$x_{c'}(q_{c,\tau}) = \begin{cases} 1, & \text{if task } q_{c,\tau} \text{ is scheduled to data center } c', \\ 0, & \text{otherwise.} \end{cases} \tag{1}$$

Naturally, a task can only be scheduled to a single data center, i.e., $\sum_{c' \in \mathcal{C}} x_{c'}(q_{c,\tau}) \leq 1$.

Given the computation requirement $l_{c,\tau}$ of task $q_{c,\tau}$, the execution time is:

$$T^{exe}(q_{c,\tau}) = \sum_{c' \in \mathcal{C}} x_{c'}(q_{c,\tau}) \cdot \frac{l_{c,\tau}}{\delta_{c',t}}, \tag{2}$$

where $\delta_{c,t}$ denotes the available resources in the data center c, which keeps constant during time slot t.

To save costs, service providers may only rent limited computing resources in each data center. Hence, the computing tasks have to queue up for execution. Let $f_{c,\tau}$ denote the workload of data center c in time instant τ, measured by the total load of tasks waiting to be executed. The queue time $T^{que}(q_{c,\tau})$ of task $q_{c,\tau}$ is defined as:

$$T^{que}(q_{c,\tau}) = \sum_{c' \in \mathcal{C}} x_{c'}(q_{c,\tau}) \cdot \frac{f_{c,\tau}}{\delta_{c',t}}, \tag{3}$$

which is the time span from the task arrival to the start of task execution.

If a task $q_{c,\tau}$ is scheduled from a busy data center c to another idle data center c', it involves an additional communication overhead between c and c'. Let $B_{c,c'}$ and $d_{c,c'}$ denote the bandwidth and the round-trip delay between c and c', respectively. The time for transmission consists of the network delay and the data sending time [31]. Actually, compared to the raw data of the task, the data volume of the computing results is trivial. Hence, the sending time of the computing results from c' to c is usually negligible [32]. Depending on where the task is scheduled, we can represent the transmission time as follow:

$$T^{com}(q_{c,\tau}) = \sum_{c' \in \mathcal{C}} x_{c'}(q_{c,\tau}) \cdot \left(d_{c,c'} + \frac{\beta_{c,\tau}}{B_{c,c'}} \right), \tag{4}$$

where $\frac{\beta_{c,\tau}}{B_{c,c'}}$ is the time for sending the raw data (i.e., $\beta_{c,\tau}$) of the task from c to c'.

The makespan of a computing task in the data center is typically defined as the time interval from task arrival to result departure, which is composed of the execution time for task computing, the queue time for waiting to be executed, and the transmission time for data transferring [29]. Thereby, the makespan of task $q_{c,\tau}$ is calculated by:

$$T^{mak}(q_{c,\tau}) = T^{exe}(q_{c,\tau}) + T^{que}(q_{c,\tau}) + T^{com}(q_{c,\tau}), \tag{5}$$

where $T^{exe}(q_{c,\tau})$, $T^{que}(q_{c,\tau})$ and $T^{com}(q_{c,\tau})$ represent the execution time, the queue time, and the communication time, respectively. To avoid task time out, the task makespan is expected to be smaller than the maximum tolerant delay, that is, $T^{mak}(q_{c,\tau}) \leq Z$.

The data transmission between data centers would impose an additional cost on the service providers. Typically, the cost of transmission is charged by the data volume. Due to the different distances between data centers, the unit price denoted by $p^{com}_{c,c'}$ is generally various. Hence, the cost for scheduling task $q_{c,\tau}$ from c to c' is represented by:

$$V^{com}(q_{c,\tau}) = \sum_{c' \in \mathcal{C}} x_{c'}(q_{c,\tau}) \cdot p^{com}_{c,c'} \cdot \beta_{c,\tau}. \tag{6}$$

From the service providers' perspective, we usually expect to minimize the transmission cost while making the task makespan meet the maximum tolerant time.

C. Resource Scaling Model

When the service provider rents too few resources, both the execution time and queue time of tasks will increase, which may degrade the service quality. Whereas, renting too many resources in the data center leads to waste, and incurs additional costs. Actually, service providers often have strict budget controls to ensure their actual costs meet financial plans. Therefore, we need to scale the rented resources dynamically while making a trade-off between the task makespan and the total cost.

At the beginning of each time slot t, each data center makes a resource scaling decision independently, which keeps unchanged during this time slot. We select the resource instance of the same configuration in different data centers as the minimum unit of resource scaling. Resource instances are usually charged by the using time and different data centers have different prices. Let $n_{c,t}$ denote the number of resource instances rented in the data center c in time slot t. Accordingly, the available resources in data center c defined in (2) can be rewritten as $\delta_{c,t} = \xi \cdot n_{c,t}$, where ξ is the amount of computing resources of an instance. For the data center c, the resource rental cost in time slot t is

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:09:31 UTC from IEEE Xplore. Restrictions apply.

defined as:

$$V_{c,t}^{ren} = n_{c,t} \cdot p_c^{ren} \cdot \Delta t, \tag{7}$$

where p_c^{ren} is the rental price of a resource instance.

In time slot t, the average workload is represented by

$$f_{c,t} = \frac{\sum_{\tau \in [t,t+1)} f_{c,\tau}}{\sum_{\tau \in [t,t+1)} \sum_{c' \in \mathcal{C}} x_c(q_{c',\tau})}, \tag{8}$$

where the numerator is the total workload and the denominator is the number of task arrived.

The average resource utilization in data center c can be represented by:

$$U_{c,t} = \frac{1}{\Delta t} \sum_{c' \in \mathcal{C}} \sum_{\tau \in [t,t+1)} x_c(q_{c',\tau}) \cdot T^{exe}(q_{c',\tau}). \tag{9}$$

Notably, the value of $U_{c,t}$ may be greater than 1, when available resources are not enough to complete all the computing tasks in time slot t. In this situation, the service is heavily overloaded. Since $U_{c,t}$ is an average value of the resource utilization in a time slot, we usually expect $U_{c,t}$ to be much smaller than 1 [26], in order to avoid task time out caused by the bursty arrival of tasks. In practice, we have $U_{c,t} \leq U$, where U is usually set to 0.8.

D. Joint Optimization Problem

By jointly considering task scheduling and resource scaling, our objective is to minimize the total costs used for data transmission among data centers (for task scheduling) and resource rental (for resource scaling) while avoiding task time out. Let T denote the time duration of the service running. We formulate the joint optimization problem as follows:

$$\min_{x,n} \quad \sum_{c \in \mathcal{C}} \sum_{t \in T} V_{c,t}^{ren} + \sum_{c \in \mathcal{C}} \sum_{\tau \in T} \sum_{c' \in \mathcal{C}} x_{c'}(q_{c,\tau}) \cdot V^{com}(q_{c,\tau}) \tag{10a}$$

$$\text{s.t.} \quad x_{c'}(q_{c,\tau}) \in \{0,1\}, \tag{10b}$$

$$\sum_{c' \in \mathcal{C}} x_{c'}(q_{c,\tau}) \leq 1, \tag{10c}$$

$$T^{mak}(q_{c,\tau}) \leq Z, \tag{10d}$$

$$U_{c,t} \leq U, \quad \forall t, \tau \in T, \quad \forall c, c' \in \mathcal{C}, \tag{10e}$$

where the constraints (10b) and (10c) ensure that the decision of task scheduling is valid, the constraint (10d) avoids task time out, and the constraint (10e) prevents service overload.

IV. Two-Timescale Joint Optimization

To address the above problem, we decompose the joint optimization problem into two sub-problems and then propose a two-timescale optimization framework.

A. Two-Timescale Problem Decomposition

As shown in (10), the resource rental cost $V_{c,t}^{ren}$ of the multi-data center system is only related to the resource scaling decision variable n_c, while the data transmission cost $V^{com}(q_{c,\tau})$ is related to the task scheduling decision variable $x_{c'}(q_{c,\tau})$ and the amount of the data $\beta_{c,\tau}$ to be scheduled. Compared to the timescale of task scheduling, the timescale of resource scaling is much longer, i.e., $\Delta\tau \ll \Delta t$, as aforementioned in Section III-B. In light of this principle, we can decompose the joint optimization problem into two sub-problems: 1) a short-timescale task scheduling problem; 2) a long-timescale resource scaling problem.

1) Task Scheduling Problem: In order to take full use of the distributed resources, tasks are scheduled among the data centers according to their workloads. The tasks are scheduled sequentially based on their arrival time. As the tasks arrive randomly, the scheduling interval, i.e., $\Delta\tau$, is variable. Since the resources are scaled once at the beginning of each time slot, the available resources remain constant during each time slot. In a short timescale, the first item of the joint optimization problem in (10) is a constant. Hence, the short-timescale task scheduling problem is formulated as:

$$\min_{x} \quad \sum_{c \in \mathcal{C}} \sum_{\tau \in [t,t+1)} \sum_{c' \in \mathcal{C}} x_{c'}(q_{c,\tau}) \cdot V^{com}(q_{c,\tau})$$

$$\text{s.t.} \quad \text{(10b), (10c), (10d).} \tag{11}$$

2) Resource Scaling Problem: In order to reduce the rental cost, the rented resources are expected to be matched with the workload. In a long timescale, based on the task scheduling $x_{c'}(q_{c,\tau}), \forall c \in \mathcal{C}, \tau \in [t-1, t)$ in last time slot $t-1$, the second item of the joint optimization problem in (10) is a constant. Hence, the long-timescale resource scaling problem can be formulated as:

$$\min_{n} \quad \sum_{c \in \mathcal{C}} \sum_{t \in T} V_{c,t}^{ren}$$

$$\text{s.t.} \quad \text{(10e).} \tag{12}$$

Since each sub-problem can be solved independently in two timescales, such decomposition can significantly reduce the complexity of problem solving.

B. Two-Timescale Joint Optimization Algorithm

To address the two sub-problems, we propose a two-timescale joint optimization framework as shown in Fig. 2. In a distributed multi-data center system, each data center acts as an agent to decide the resource scaling and the task scheduling independently. In a short timescale, the tasks are scheduled among the data centers according to the workloads and the task attributes. The short-time task scheduling can promptly but temporally relieve the bursty arrivals of computing tasks. In a long timescale, the resources are scaled to match with the long-term changes of workloads. The long-time resourc scaling changes the resource distribution among the data centers, which could in turn impact the short-time task scheduling. The resource scaling and task scheduling are carried out alternately to adapt to the spatio-temporal dynamic changes of workloads in the multi-data center system.

1) Short-Timescale Task Scheduling: In a short timescale, we decide where to schedule the tasks based on the state of workload

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:09:31 UTC from IEEE Xplore. Restrictions apply.

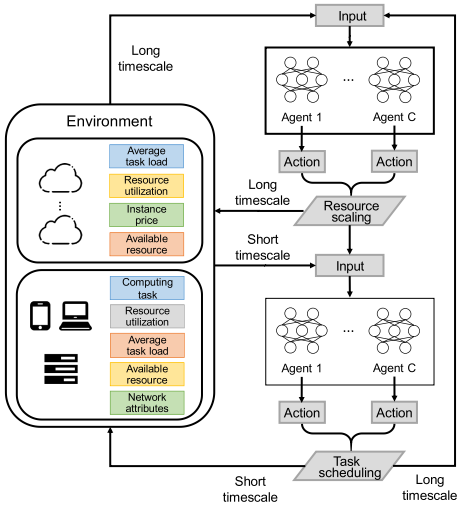


Fig. 2. Two-timescale joint optimization framework.

and task arrival. Because the state transition only depends on the current state and scheduling action, which is consistent with the Markov property, we can model the short-timescale task scheduling problem as a Markov Decision Process (MDP). The state, action, and reward in a short timescale are defined as follows.

1) State: Considering the connectivity between distributed data centers, we use a directed graph $\mathcal{G} = \{\mathcal{C}, \mathcal{E}\}$ to denote the multi-data center system, where the data centers compose a vertex set $\mathcal{C}$, and the connections between them compose an edge set $\mathcal{E}$. The attributes of each vertex c are represented by $\{\delta_{c,\tau}, f_{c,\tau}\}$, which characterizes the current state of the available resources and the workload in data center c. The attributes of each edge $e_{c,c'} \in \mathcal{E}$ are represented by $\{B_{c,c'}, d_{c,c'}\}$, which characterizes the bandwidth and delay of the connection from c to c'. Since both the task load and the network condition are time-varying, $\mathcal{G}$ is updated with time. However, updating $\mathcal{G}$ in a short timescale needs to collect and share the state of each distributed data center frequently, which would cause vast communication overheads. In practice, we usually update $\mathcal{G}$ in a long timescale using the average value of a time slot, that is, $\mathcal{G}_\tau$ and $\delta_{c,\tau}$ remain unchanged for any $\tau \in [t, t+1)$. Thereby, we use $\mathcal{G}_t$ and $\delta_{c,t}$ to replace $\mathcal{G}_\tau$ and $\delta_{c,\tau}$, respectively. In time instant $\tau \in [t, t+1)$, we can easily get the task attributes $\{l_{c,\tau}, \beta_{c,\tau}\}$, the local state $\{\delta_{c,t}, f_{c,\tau}\}$ and the shared global state $\mathcal{G}_t$. Hence, when scheduling task $q_{c,\tau}$, the state is defined as:

$$\boldsymbol{s}^S_{c,\tau} = \{l_{c,\tau}, \beta_{c,\tau}, \delta_{c,t}, f_{c,\tau}, \mathcal{G}_t\}. \tag{13}$$

2) Action: The action of scheduling task $q_{c,\tau}$ is defined as:

$$\boldsymbol{a}^S_{c,\tau} = \{x_1(q_{c,\tau}), \ldots, x_{c'}(q_{c,\tau}), \ldots, x_C(q_{c,\tau})\}. \tag{14}$$

Naturally, the task scheduling action should be consistent with the constraints in (10b) and (10c).

3) Reward: The goal of task scheduling is to minimize the cost while meeting the requirement of the task makespan. Hence, the reward of short-timescale task scheduling is defined as:

$$R^S_{c,\tau} = -V^{com}(q_{c,\tau}) - \omega_1 P_1(q_{c,\tau}), \tag{15}$$

where $P_1(q_{c,\tau})$ is the penalty for task timeout, and ω_1 is the coefficient of penalty. The penalty is defined as

$$P_1(q_{c,\tau}) = \begin{cases} 0, & T^{mak}(q_{c,\tau}) \le Z, \\ e^{(\frac{T^{mak}(q_{c,\tau})-Z}{Z})^2}, & T^{mak}(q_{c,\tau}) > Z. \end{cases} \tag{16}$$

Naturally, there is no penalty for the tasks completed on time. However, when the task makespan violates the maximum tolerant delay Z, the penalty grows exponentially, punishing the actions that lead to the task timeout.

Each data center c has to decide its own task scheduling action $\boldsymbol{a}^S_{c,\tau}$ according to the states $\boldsymbol{s}^S_{c,\tau}$. Let π^S_c denote the scheduling strategy, which maps a state to a legitimate action.

$$\boldsymbol{a}^S_{c,\tau} = \pi^S_c(\boldsymbol{s}^S_{c,\tau}), \tag{17}$$

For this sub-problem, our goal is to find an optimal scheduling strategy that maximizes the overall reward of all data centers.

2) Long-Timescale Resource Scaling: In a long timescale, service providers need to scale the rented resources according to the dynamic changes of workloads in each data center. Since the future state of the workload is only related to its current state and the scaling action, which is consistent with the Markov property, this problem can also be modeled as an MDP. We define state, action and reward in a long timescale as follows:

1) State: To accommodate the long-term changes of computing tasks, each data center has to scale the resources based on the resource usage and the workload. Hence, when scaling the resources, the state of any data center c in time slot t is defined as:

$$\boldsymbol{s}^L_{c,t} = \{\delta_{c,t}, U_{c,t}, f_{c,t}, \mathcal{G}_t\}, \tag{18}$$

which jointly considers the available resources $\delta_{c,t}$, the average resource utilization $U_{c,t}$, the average workload $f_{c,t}$ and the global state $\mathcal{G}_t$.

2) Action: Cloud providers usually provide some VMs or some containers with certain computing power as a resource instance to the tenant, which can be used as the basic unit of resource scaling. The action of resource scaling in data center c is defined as:

$$\boldsymbol{a}^L_{c,t} \in \{-1, 0, +1\}, \tag{19}$$

where -1 means to reduce a resource instance, 0 means to take no change and $+1$ means to increase a resource instance. In practice, we can adjust the computing resources of a single instance to accommodate the variation of the workload within a time slot.

3) Reward: The optimization goal of resource scaling is to minimize the resource rental cost while preventing service overload. The reward of long-timescale resource scaling is defined as:

$$R^L_{c,t} = -V^{ren}_{c,t} - \omega_2 P_2(t), \tag{20}$$

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:09:31 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Two-Timescale Joint Optimization Algorithm.

Input: A sequence of tasks $\{q_{c,\tau}\}$.
Output: Task scheduling $\boldsymbol{a}^S_{c,\tau}$, resource scaling $\boldsymbol{a}^L_{c,\tau}$

1 **Initialization:** Z, U;
2 **for** $t = \{1, 2, \cdots\}$ **do**
3 **for** *all data centers* c *in* $\mathcal{C}$ **do**
4 **for** *all* $\tau \in t$ **do**
5 Get the states $\boldsymbol{s}^S_{c,\tau}$ of data center c;
6 Calculate $\boldsymbol{a}^S_{c,\tau}$ based on Eq. (17);
7 Execute task scheduling decision $\boldsymbol{a}^S_{c,\tau}$;
8 **end**
9 Get the states $\boldsymbol{s}^L_{c,t}$ of data center c;
10 Calculate $\boldsymbol{a}^L_{c,t}$ based on Eq. (22);
11 Execute resource scaling decision $\boldsymbol{a}^L_{c,t}$;
12 **end**
13 **end**

where $P_2(t)$ is the penalty for service overload and ω_2 is the coefficient of penalty. The penalty is defined as:

$$P_2(t) = \begin{cases} 0, & U_{c,t} \leq U, \\ e^{\left(\frac{U_{c,t}-U}{U}\right)^2}, & U_{c,t} > U. \end{cases} \tag{21}$$

Naturally, there is no penalty if the resource utilization can accommodate the task loads, $U_{c,t} \leq U$. Otherwise, if the resource utilization $U_{c,t}$ exceeds the predefined threshold U, we have to impose a penalty that grows exponentially with the increase of the difference between $U_{c,t}$ and U.

Each data center c has to decide its own scaling action $\boldsymbol{a}^L_{c,t}$ according to the states $\boldsymbol{s}^L_{c,t}$. Let π^L_c represent the scaling strategy, which maps a state to a legitimate action:

$$\boldsymbol{a}^L_{c,t} = \pi^L_c(\boldsymbol{s}^L_{c,t}). \tag{22}$$

For this sub-problem, our goal is to find an optimal scaling strategy that maximizes the overall reward of all data centers.

The two-timescale joint optimization algorithm can be summarized in Algorithm 1. The tasks are scheduled among data centers upon arrival based on a certain scheduling strategy (line 4-8). In each time slot t, we scale the number of resource instances used for task computing (line 9-12). The remaining problem is to establish a decision-making model that gives a proper scaling strategy π^L_c and a proper scheduling strategy π^S_c, which are elaborated in the next section.

V. Multi-Agent Deep Reinforcement Learning Based Decision Making

In this section, we propose a multi-agent deep reinforcement learning based decision model which can be applied to both resource scaling and task scheduling. Then, we adopt a DGCN to characterize the graph structured state of the distributed multi-data center system. Finally, we give an *Asynchronous Advantage Actor-critic* (A3C) based parallel model training method and perform a complexity analysis of the algorithm.

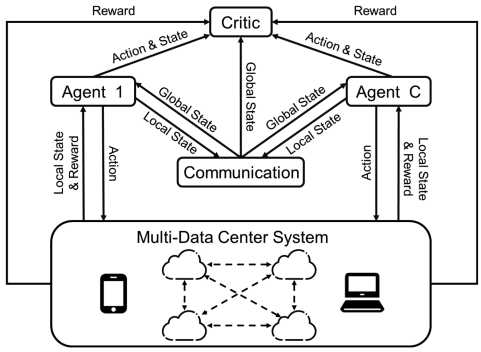


Fig. 3. Multi-agent actor-critic decision model.

A. Multi-Agent Actor-Critic Decision Model

The multi-agent actor-critic decision model of joint optimization is presented in Fig. 3, which consists of three modules: actor, critic, and communication.

1) Actor: Each data center acts as an agent with an actor network that learns a policy function mapping a specific state to a legitimate action with the aim of maximizing the overall reward. Since the tasks can be migrated among the data centers, the decision making should take both the state of local task arrivals and the state of global load distribution into account.

2) Critic: The critic network aims to learn a state-action value function, which estimates the reward for a given state-action pair. The critic output is used for guiding the training of actors. Due to the high dimensionality of both state and action space, the convergence speed of model training is slow in practice. In order to speed up the training process, the state-action value function is decomposed into a state-value function and an advantage function using A3C.

3) Communication: Since all agents are expected to work cooperatively, the global state $\mathcal{G}_t$ needs to be shared among the agents by the communication module. The graph structured $\mathcal{G}$ cannot be handled directly by the actor network. Here, we use a graph convolutional neural network for capturing the spatial features of the graph $\mathcal{G}$, which is elaborated in Section V-B.

B. DGCN Based Global State Representation

The data centers are connected to each other, forming a irregular graph, where the attributes of each edge, i.e., the bandwidth and delay between data centers, are time-varying. The decision making should consider the graph structure, including both the attributes of each node and the connections between nodes. The traditional neural networks such as Multi-Layer Perceptron (MLP) and GRU, are widely used to process image data and sequence data, however they can hardly to characterize the graph structured connection relationship between data centers [33]. To solve this problem, we use Graph Neural Network (GNN) to extract structural features in the multi-data center system. Since $\mathcal{G}$ is a bidirectional graph, we adopt a DGCN to represent the spatial characteristics of global state $\mathcal{G}_t$ as elaborated in Fig. 4. The attributes of all vertexes form a state matrix X, and the attributes of all edges form a weight matrix W. According to the

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:09:31 UTC from IEEE Xplore. Restrictions apply.

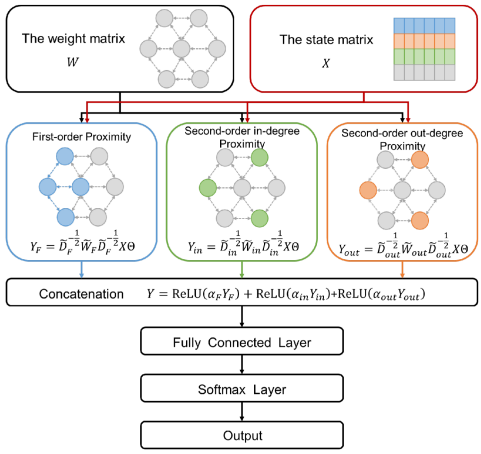


Fig. 4. Global state representation process using DGCN.

connection relationship of the graph, we calculate the first-order proximity matrix W_F as follows:

$$W_F(i,j) = \begin{cases} 1, & \text{if } w_{i,j} \neq 0 \text{ or } w_{j,i} \neq 0, \\ 0, & \text{otherwise}, \end{cases} \tag{23}$$

where $i, j \in \mathcal{C}$ denote the vertexes, $w_{i,j}, w_{j,i} \in W$.

Meanwhile, the second-order proximity matrix, including in-degree proximity matrix W_{in} and out-degree proximity matrix W_{out}, can be calculated by:

$$W_{in}(i,j) = \sum_{k \in \mathcal{C}} \frac{w_{k,i} w_{k,j}}{\sum_{v \in \mathcal{C}} w_{k,v}}, W_{out}(i,j) = \sum_{k \in \mathcal{C}} \frac{w_{i,k} w_{j,k}}{\sum_{v \in \mathcal{C}} w_{v,k}}, \tag{24}$$

where, $W_{in}(i,j)$ sums up the normalized weights of edges which array to both i and j, which can reflect the similarity of the in-degree between vertex i and j. Similarly, $W_{out}(i,j)$ measures the second-order out-degree proximity by summing the normalized weights of the edges from i and j. Notably, the three proximity matrices above are symmetric.

The degree matrix D_* corresponding to each proximity matrix W_* can be calculated as $D_*(i,i) = \sum_{j \in \mathcal{C}} W_*(i,j)$, where $* \in \{F, in, out\}$, and D_* is a diagonal matrix. Afterwards, based on the Chebyshev polynomials, the convolution matrices of the above three proximity matrices are calculated by:

$$Y_* = \widetilde{D}_*^{-\frac{1}{2}} \widetilde{W}_* \widetilde{D}_*^{-\frac{1}{2}} X \Theta, \tag{25}$$

where $\widetilde{W}_* = W_* + \lambda I$, $\widetilde{D}_* = D_* + \lambda I$, Θ is the linear transformation matrix.

Finally, we get the spatial feature of the graph $\mathcal{G}$ by fusing the above convolution outputs:

$$Y = Con\left(\text{ReLU}(\alpha_F Y_F), \text{ReLU}(\alpha_{in} Y_{in}), \text{ReLU}(\alpha_{out} Y_{out})\right), \tag{26}$$

where $Con(\cdot)$ denotes the concatenation operation, α_F, α_{in}, α_{out} are the coefficients, and $\text{ReLU}(\cdot)$ is the activation function. Instead of the raw graph structured $\mathcal{G}$, the communication module will share the feature vector Y as the global state among all agents.

C. A3C-Based Model Training

To speed up the convergence of model training, we employ an A3C-based scheme to train the above multi-agent deep reinforcement learning model. With A3C, multiple actor-critic networks (workers) run in their own independent environments, and the global shared network (master) updates network parameters asynchronously with each worker. Such master-worker structure makes it possible to perform the training process in parallel.

Since both the short-timescale task scheduling and long-timescale resources scaling use the same actor-critic decision model, we use unified notations of μ, $\boldsymbol{s}$, $\boldsymbol{a}$, R and π to refer to the decision time, state, action, reward and policy defined in Section IV-B. For each agent c, an actor network is used to estimate the policy function, which is represented by:

$$\pi_c\left(\boldsymbol{s}_\mu; \theta_c^a\right) = \varphi_c^a\left(\boldsymbol{s}_\mu; \theta_c^a\right), \tag{27}$$

where $\varphi_c^a(\cdot)$ is the actor network of agent c, and θ_c^a is the set of network parameters.

Using A3C, the state-action value function $Q(\boldsymbol{s}_\mu, \boldsymbol{a}_\mu)$ is decomposed into a state value function $V(\boldsymbol{s}_\mu)$ and an advantage function $A(\boldsymbol{s}_\mu, \boldsymbol{a}_\mu)$, i.e., $Q(\boldsymbol{s}_\mu, \boldsymbol{a}_\mu) = A(\boldsymbol{s}_\mu, \boldsymbol{a}_\mu) + V(\boldsymbol{s}_\mu)$. The state value function is estimated by the critic network as follow:

$$V\left(\boldsymbol{s}_\mu; \theta^r\right) = \varphi^r\left(\boldsymbol{s}_\mu; \theta^r\right), \tag{28}$$

where $\varphi^r(\cdot)$ is the critic network, and θ^r is the set of network parameter. According to the Bellman equation, the state-action value can be calculated by $Q(\boldsymbol{s}_\mu, \boldsymbol{a}_\mu) = R_\mu + \gamma V(\boldsymbol{s}_{\mu+1}; \theta^r)$. In a multi-agent system, R_μ is the sum of the individual rewards of all agents, i.e., $R_\mu = \sum_{c \in \mathcal{C}} R_{c,\mu}$. Therefore, the advantage function can be estimated by the K-step Generalized Advantage Estimation (GAE):

$$A\left(\boldsymbol{s}_\mu, \boldsymbol{a}_\mu; \theta_c^a, \theta^r\right) = \sum_{i=0}^{K-1} (\gamma\lambda)^i \Phi_{\mu+i}, \tag{29}$$

where $\Phi_{\mu+i} = R_{\mu+i} + \gamma V[\boldsymbol{s}_{\mu+i+1}; \theta^r] - V[\boldsymbol{s}_{\mu+i}; \theta^r]$, and $\lambda \in (0,1)$ measures the compromise between bias and variance.

The actor loss function of agent c is defined as:

$$L_{\theta_c^a,\mu} = \log\pi\left(\boldsymbol{a}_\mu | \boldsymbol{s}_\mu; \theta_c^a\right) A\left(\boldsymbol{s}_\mu, \boldsymbol{a}_\mu; \theta_c^a, \theta^r\right) + \rho H\left(\pi\left(\cdot | \boldsymbol{s}_\mu; \theta_c^a\right)\right), \tag{30}$$

where $H(\cdot)$ is the entropy regularization term with decaying parameter ρ.

The critic loss function is defined as the K-step Time-Difference error of the state value function:

$$L_{\theta^r,\mu} = \frac{1}{K} \sum_{i=0}^{K-1} \{\Phi_{\mu+i}\}^2. \tag{31}$$

The master uses RMSProp to update the parameters $\boldsymbol{\theta}^a$ and θ^r from all workers asynchronously:

$$g = \zeta g + (1-\zeta)\Delta\theta^2, \quad \theta \longleftarrow \theta - \eta \frac{\Delta\theta}{\sqrt{g+\epsilon}}, \tag{32}$$

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:09:31 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: Model Training Process.

Input: State $\boldsymbol{s}_{c,\mu}$; update step K; maximum number of episode E;
Output: Optimal parameters of actor $\theta_c^a, \forall c \in \mathcal{C}$;
1 **Initialization:** Actor network parameters $\boldsymbol{\theta}^a$, critic network parameters θ^r, shared networks parameters $\widetilde{\boldsymbol{\theta}}^a$, $\widetilde{\theta}^r$;
2 **for** *each training worker* **do**
3 **for** $episode = \{1, \cdots, E\}$ **do**
4 Update the global state $\mathcal{G}_t$;
5 Synchronize parameters from the shared network $\boldsymbol{\theta}^a = \widetilde{\boldsymbol{\theta}^a}$, $\theta^r = \widetilde{\theta^r}$;
6 **for** $k = \{1, \cdots, K\}$ **do**
7 **for** $\forall c \in \mathcal{C}$ **do**
8 Obtain the state $\boldsymbol{s}_{c,\mu}$;
9 The actor outputs the action $\boldsymbol{a}_{c,\mu}$;
10 Obtain the reward $R_{c,\mu}$ and the next state $\boldsymbol{s}_{c,\mu+1}$;
11 Record $\{\boldsymbol{s}_{c,\mu}, \boldsymbol{a}_{c,\mu}, R_{c,\mu}, \boldsymbol{s}_{c,\mu+1}\}$;
12 **end**
13 **end**
14 **for** $\forall c \in \mathcal{C}$ **do**
15 Evaluate the loss function $L_{\theta_c^a,\mu}$ of the actor c by Eq. (30);
16 Asynchronously update the shared network parameters $\boldsymbol{\theta}_c^a$ by Eq. (32);
17 **end**
18 Evaluate the loss function $L_{\theta^r,\mu}$ of the critic by Eq. (31);
19 Asynchronously update the shared network parameters θ^r by Eq. (32);
20 **end**
21 **end**

where $\Delta\theta$ is the network gradient of loss function, g is the elementwise square gradient, ζ is the momentum, and η is the learning rate.

Based on the above discussion, the training process of the multi-agent deep reinforcement learning algorithm can be summarized as Algorithm 2. In each episode, each training worker updates its own network using the parameters of the shared network from the master (line 4-5). Each trained worker runs in an independent environment and collects experience $\{\boldsymbol{s}_{c,\mu}, \boldsymbol{a}_{c,\mu}, R_{c,\mu}, \boldsymbol{s}_{c,\mu+1}\}$ (line 6–13). The above process is performed K rounds in an episode. Finally, the loss function is calculated through the obtained experience, and the parameters of the shared model are updated (line 14–19).

D. Computational Complexity Analysis

When evaluating the complexity of the algorithm, we need to consider the forward propagation complexity of the network, including the state representation model and the decision making model.

State Representation: The complexity of state representation based on DGCN model is $O(C^2 A_{in}^{gcn} + C A_{in}^{gcn} H)$, where A_{in}^{gcn} is the size of the DGCN input dimension and H is the size of the output dimension. Then, the output of DGCN goes through two FC layers, and the complexity is $O(H^2)$. We can obtain the complexity of the DGCN is $O(C^2 A_{in}^{gcn} + C A_{in}^{gcn} H + H^2)$.

Decision Making: The decision making process consists of C actors, each of which consists of three FC layers. The complexity of the actors is $O(C(A_{in}^{a} H + H^2 + HC))$, where A_{in}^{a} is the size of the actor input dimension.

Hence, the total time complexity in the decision process is $O(C^2(A_{in}^{gcn} + H) + CH(A_{in}^{gcn} + A_{in}^{a} + H) + H^2)$.

VI. Experiment Results and Analysis

In this section, we conduct experiments to verify the effectiveness of our proposed *2TAD* algorithm and perform performance analysis.

A. Simulation Setup

To ensure the practicality of the simulation, we select 10 data centers of Amazon Cloud as experimental nodes, i.e., Northern Virginia, Oregon, Sao Paulo, Singapore, Frankfurt, Hong Kong, Tokyo, Seoul, Paris and London. During the experiment, we select the commonly used m5.24xlarge type VM in the Amazon data center, and use 10 VMs as the basic unit of resource scaling, that is, a resource instance. At the initial moment, the number of resource instances in the 10 data centers are set to $\{6, 6, 6, 7, 7, 7, 8, 8, 8, 8\}$ respectively. The network latency (acquired from [34]), VM price, data transmission price, and network bandwidth between data centers come from real data in Amazon Cloud. The major configurations of these data centers are summarized in Table III.

In our experiment, the time interval of each slot t is set to 1 minute. The task arrival in each data center obeys a non-homogeneous Poisson distribution which is widely used in the literature [35], the intensity $\lambda(t)$ of which is set to a sinusoidal function. In this way, the workload is time-varying. Meanwhile, in order to stagger the workload peaks of different data centers, there is a phase shift in the sine function of different data centers. We plot the changes of workloads with an average intensity $\bar{\lambda} = 3.6/s$ in Fig. 5. We can find that the workloads of different data centers exhibit spatio-temporal dynamics. We set the CPU cycles required by the task to be linearly increased with the amount of data, which is also used in [36], i.e., $l_{c,\tau} = \alpha \cdot \beta_{c,\tau}$, where α is the computation-to-data ratio. The calculation amount of each task obeys a uniform distribution of [100, 500] MB, and $\alpha = 9900$ cycles/byte. We use the Pytorch library to implement the algorithm model of *2TAD*, and the neural network architecture is shown in Table IV. We train the DRL-based algorithms mentioned above with 100 episodes, and each episode lasts for an hour.

The following three state-of-the-art algorithms are implemented as baselines for performance comparison:

- *VDN: VDN* [13] employs a distributed reinforcement learning based algorithm to schedule the tasks. However, *VDN* only considers the task scheduling, but rarely considers the resource scaling.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:09:31 UTC from IEEE Xplore. Restrictions apply.

TABLE III
CONFIGURATION OF TEN DATA CENTERS

Data Center c	Northern Virginia	Oregon	Sao Paulo	Singapore	Frankfurt	Hong Kong	Tokyo	Seoul	Paris	London
VM Price ($ per hour)	4.608	4.608	7.344	5.76	5.52	6.366	5.952	5.664	5.376	5.328
Data Transmission Price ($ per GB)	0.02	0.02	0.138	0.09	0.02	0.09	0.09	0.08	0.02	0.02
Network Bandwidth (GB)	10	10	10	10	10	10	10	10	10	10

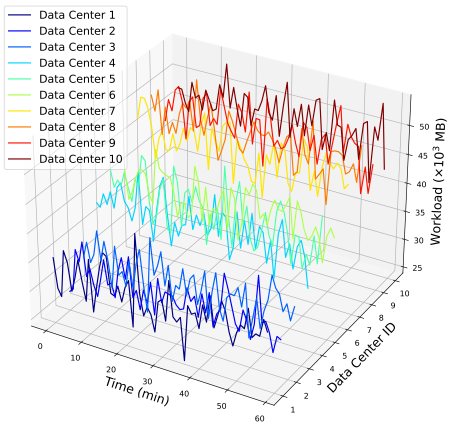


Fig. 5. Data center workload with spatio-temporal dynamics.

TABLE IV
NEURAL NETWORK STRUCTURE SETTING

	Model	Network Structure
State Representation	DGCN	3 Graph Convolution Layers
		2 Fully Connected Layers
Task Scheduling	Actor	3 Fully Connected Layers
	Critic	3 Fully Connected Layers
Resource Scaling	Actor	3 Fully Connected Layers
	Critic	3 Fully Connected Layers

- *HANSEL: HANSEL* [21] proposes an adaptive resource scaling algorithm based on distributed reinforcement learning. However, it only considers the problem of resource scaling, rather than the problem of task scheduling.
- *RALBA:* We implement a heuristic joint optimization algorithm by combining the *Resource-Aware LoadBalancing Algorithm* (*RALBA*) [37] based task scheduling strategy with the threshold-based resource scaling strategy [38].
- *Exhaustive Search (ES): ES* conducts task scheduling based on an exhaustive search policy which chooses the data center with the minimum transfer cost from the ones that can complete the task on time. A threshold-based resource scaling scheme [38] is adopted by *ES*, which is widely used by commercial cloud providers. *ES* gives an upper bound of task scheduling, however, *ES* needs to know the real-time state of all agents when scheduling each task, which is impractical in a real multi-data center system.

B. Performance Metrics

To evaluate the performance of these algorithms, we define the following metrics:

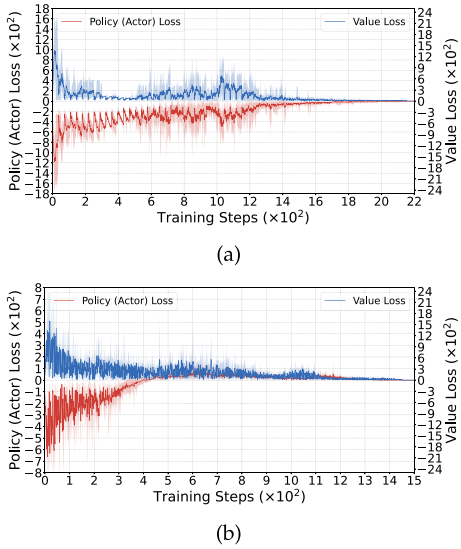


Fig. 6. The change of the loss value of actor and critic during the training process of models. (a) Resource scaling model. (b) Task scheduling model.

- *Makespan* quantifies the total time required for all data centers to complete all computing tasks as defined in (5).
- *Cost* quantifies the total cost required to complete all computing tasks as defined in (10).
- *Utilization* quantifies the ratio of CPU cycles used for task to the total cycles, which is defined as:

$$\widetilde{U} = \frac{1}{CT} \sum_{c \in \mathcal{C}} \sum_{t \in T} U_{c,t}. \tag{33}$$

  Since a too low resource utilization leads to a waste of computing resources, we expect a high resource utilization.
- *Task timeout ratio* is defined as the proportion of tasks that exceed the tolerant time.

C. Results and Analysis

The training process of *2TAD* is shown in Fig. 6. The x-axis in the figure represents the training step. The red and blue curves represent the average loss value of the actor and the critic, respectively. The shaded area is the variance range of the actual loss of multiple workers. From Fig. 6(a) and (b), we find that the loss of resource scaling converges to 0 after about 1.8×10^3 training steps, and the loss of task scheduling converges to 0 after about 1.3×10^3 training steps. This result indicates that the well-trained *2TAD* has learned a stable policy for task scheduling and resource scaling.

Fig. 7 shows the performance comparison of the five algorithms under the average intensity of 3.6/s. In Fig. 7(a), the

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:09:31 UTC from IEEE Xplore. Restrictions apply.

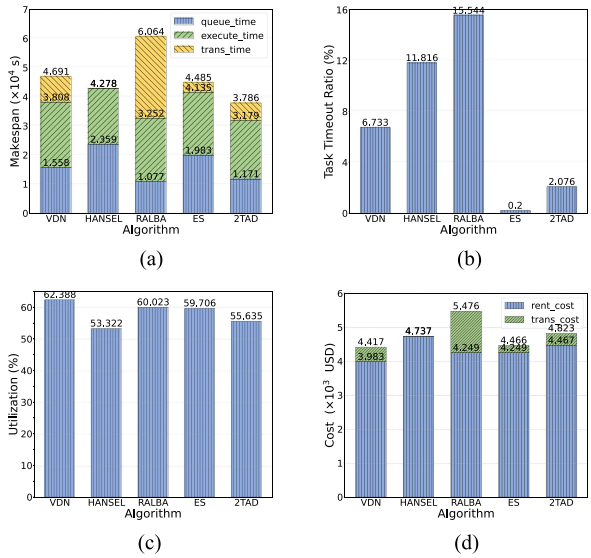


Fig. 7. Performance under dynamic task arrivals. (a) Makespan. (b) Task timeout ratio. (c) Utilization. (d) Cost.

task makespan is composed of queue time, execution time, and data transmission time. The total task makespan of *2TAD* is the smallest, i.e. about 3.8×10^4 s, followed by *HANSEL*, *ES* and *VDN* while *RALBA* suffers from the largest task makespan over 6×10^4 s. To avoid task timeout, *VDN* considers scheduling the tasks in some overloaded data centers to the idle ones, which reduces the queue time but significantly increases the data transmission time. However, *VDN* cannot reduce the task execution time without scaling up resources. By contrast, without regard to task scheduling, *HANSEL* rarely involves data transmission time. When the data center is overloaded, *HANSEL* may rent more computing resources to reduce the execution time of tasks. Due to the long time scale of resource scaling, the queue time of *HANSEL* may have a sharp increase when tasks arrive in a bursty manner. When scheduling tasks, *RALBA* only focuses on the resource utilization of data centers and ignores the network conditions between data centers, which reduces the task queuing time and execution time, but greatly increases the data transmission time. By jointly optimizing task scheduling and resource scaling, *2TAD* can effectively reduce the makespan of tasks. From Fig. 7(a), we find that the queuing time of *HANSEL* is higher than other algorithms. Compared to *HANSEL* that processes workloads in a single data center, *VDN*, *RALBA*, *ES* and *2TAD* that consider task scheduling between multi-location data centers can schedule tasks from busy data centers to idle ones, thereby reducing the queuing time in busy data centers. However, task scheduling between multiple data centers will also increase data transmission time. *RALBA* has the longest data transmission time (3.8×10^4 s), followed by *VDN* (0.9×10^4 s), *2TAD* (0.6×10^4 s) and *ES* (0.35×10^4 s).

Fig. 7(b) gives the task timeout ratio of each algorithm. *RALBA* has the highest task timeout ratio of nearly 16%, followed by *HANSEL* (nearly 12%), *VDN* (less than 6%) and *2TAD* (about 2%), and almost no task timeout occurs for *ES* (close to 0.5%). *HANSEL* can adapt to the long-term variations of workloads, but cannot have a prompt response to the bursty task arrivals, which easily leads to task timeout. By contrast, although *VDN* can alleviate the task timeout by scheduling the bursty tasks to other light-loaded data centers, it can hardly reduce the timeout ratio anymore via task scheduling when the workload exceeds the computing capacity of the whole system. *RALBA* has a great transmission time, which increases the timeout ratio of tasks. While scheduling tasks among multi-location data centers, *2TAD* takes into account the communication overhead, thus can adapt to dynamic changes in workload and effectively reduce task timeout rate through short-term task scheduling and long-term resource scaling. Since *ES* can get the real-time state of the whole system which however is infeasible in practice, the task scheduling will outperform *2TAD*, resulting in a much lower tas timeout ratio. Compared to processing workloads in a single data center alone (*HANSEL*), scheduling tasks among multi-location data centers (*VDN*, *ES* and *2TAD*) can effectively reduce the task timeout ratio.

Fig. 7(c) shows the average resource utilization of each algorithm. *VDN* has the highest resource utilization (about 62%), followed by *RALBA*, *ES* and *2TAD* (60%, 59.5% and 55%), and *HANSEL* suffers from the lowest resource utilization (close to 52%). When the workload is constant, the resource utilization is inversely proportional to the rented computing resources as defined in (9). Since the same trace of task arrivals is used for different algorithms in the evaluation, more computing resources means a lower resource utilization. As shown in Fig. 7(d), *HANSEL* rents the most resources and consumes the maximum resource rental cost, whereas *VDN* rents the least resources and consumes the minimum resource rental cost. Meanwhile, the total cost is composed of resource rental cost and data transmission cost. Although *VDN* has minimal resource rental costs, its task scheduling increases transmission costs. Compared to *HANSEL*, which only scales resources, *VDN*, *RALBA*, *ES* and *2TAD* reduce the resource rental cost, but increase the data transmission cost. Similar to data transmission time, *RALBA* has the highest data transmission cost (1.2×10^3 USD), followed by *VDN* (0.43×10^3 USD), *2TAD* (0.35×10^3 USD) and *ES* (0.22×10^3 USD). From the Fig. 7(d), we find that except for *RALBA*, there is no remarkable difference between the costs of the remaining four algorithms. The above results in Fig. 7 indicate that compared to the other four baseline algorithms, the proposed *2TAD* can effectively reduce the makespan and timeout ratio of the tasks, while consuming the similar cost. That is, *2TAD* makes a well trade-off between cost and service quality.

To verify the performance of resource scaling, we select one data center from the 10 data centers, and demonstrate the resource scaling behaviors and resource utilization changes of three typical comparison algorithms, including *VDN*, *HENSEL* and *2TAD*, in Fig. 8. Fig. 8(a) shows the changes of workload over time in Data Center 4. Fig. 8(b), (c) and (d) show the corresponding changes of the resource instances, resource utilization and task timeout ratio, respectively. We find that the resource instances of *VDN* remain unchanged, while *HENSEL* and *2TAD* dynamically adjust the number of resource instances to adapt to the dynamic of workload. The average number of

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:09:31 UTC from IEEE Xplore. Restrictions apply.

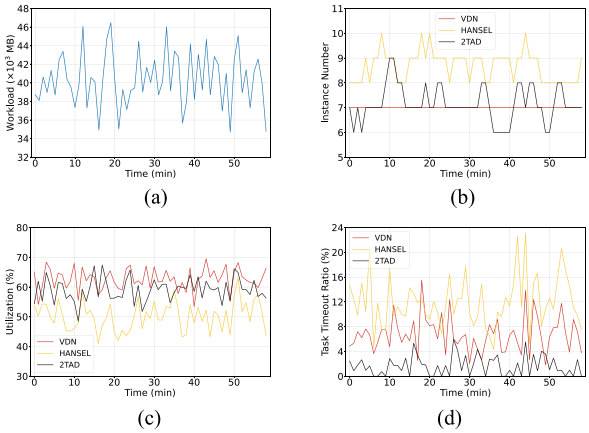


Fig. 8. The variations in workload, instance numbers, resource utilization, and task timeout ratios at Data Center 4. (a) Workload. (b) Instance number. (c) Utilization. (d) Task timeout ratio.

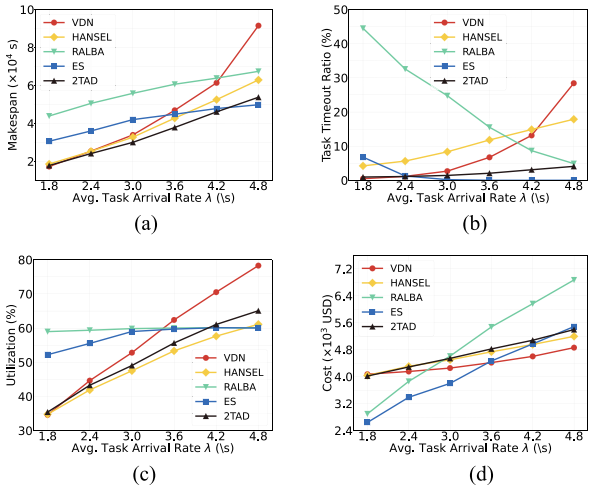


Fig. 9. Performance under different average task arrival rates. (a) Makespan. (b) Task timeout ratio. (c) Utilization. (d) Cost.

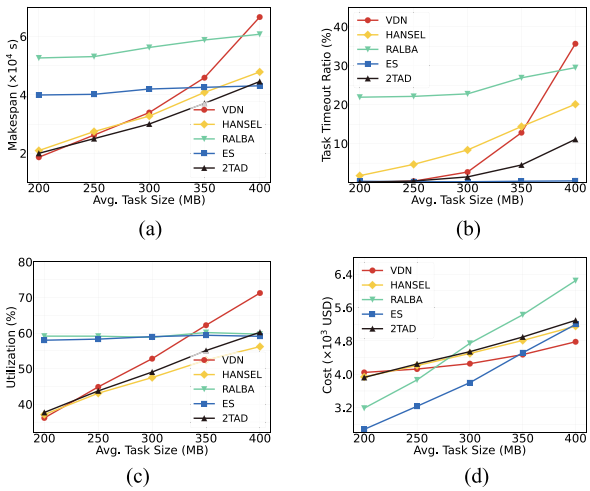


Fig. 10. Performance under different average task sizes. (a) Makespan. (b) Task timeout ratio. (c) Utilization. (d) Cost.

resource instances of *HENSEL* is larger than that of *2TAD*. Because *VDN* does not consider resource scaling, the number of resource instances always remains in the initial state. Compared to *HENSEL*, *2TAD* has smaller average number of resource instances, because it can balance the workload between data centers through task scheduling. Since *VDN* consumes the least computing resources, the corresponding resource utilization is the highest, followed by *2TAD* and *HENSEL*. Fig. 8(d) illustrates the timeout ratio of tasks submitted to Data Center 4 under the three algorithms. Among them, the average task timeout ratio of *HANSEL* is the highest, followed by *VDN* and *2TAD*. Because *HANSEL* only considers resource scaling, which can lead to task timeouts when tasks arrive in bursts. *VDN* schedules tasks among multiple data centers, which can reduce the task timeout ratio, but when all data centers are busy, a large number of task timeouts are inevitably going to occur. By jointly optimizing task scheduling and resource scaling on two time scales, *2TAD* can effectively reduce the task timeout ratio. The above experimental results indicate that resource scaling can accommodate the time-varying workloads.

In order to verify the performance of the 2TAD algorithm under different average intensity $\bar{\lambda}$, we generated a series of data sets by setting $\bar{\lambda}$ to 1.8/s, 2.4/s, 3.0/s, 3.6/s, 4.2/s and 4.8/s, while keeping other experimental parameters unchanged. We have verified the five algorithms on different datasets, and the experimental results are shown in Fig. 9. We find that the trend of the curves for *VDN*, *HANSEL* and *2TAD* is basically consistent, except for *RALBA* and *ES*. That is, the makespan, task timeout ratio, utilization and cost are all increasing with the increase of the task arrival rate. As the task arrival rate increases, more tasks will wait in the queue for processing, which naturally increases the queue time as well as the task makespan. Accordingly, a larger task makespan inevitably increases the chance of task timeout. Meanwhile, since more tasks will increase the workload of the data center, more resources need to be rented to compute the tasks and more data need to be transmitted, that is, both the resources rental cost and the data transmission cost are increased. By contrast, the task timeout ratios of *RALBA* and *ES* decrease with the increase of the task arrival rate. The threshold-based resource scaling algorithm adopted by *RALBA* and *ES* is sensitive to the workload changes. Once the resource utilization rises over a predefined threshold, more resources will be rented by *RALBA* and *ES*, leading to a decrease in the task timeout ratio but an increase in the total cost. From Fig. 9, we also find that the task timeout ratio of *VDN* and *HANSEL* increases rapidly with the growth of task arrival rate, which seriously affects the quality of service. This result indicates that task scheduling and resource scaling alone cannot adapt to the variations in workload.

In order to show the performance comparison under different task data sizes, in the evaluation, we set the average data size to increase from 200 MB to 400 MB with a step of 50 MB. The amount of data for each task obeys a uniform distribution, where the actual data size fluctuates around an average value within a range of ± 200 MB. The results are plotted in Fig. 10. We find that the trend of the curves for *VDN*, *HANSEL* and

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:09:31 UTC from IEEE Xplore. Restrictions apply.

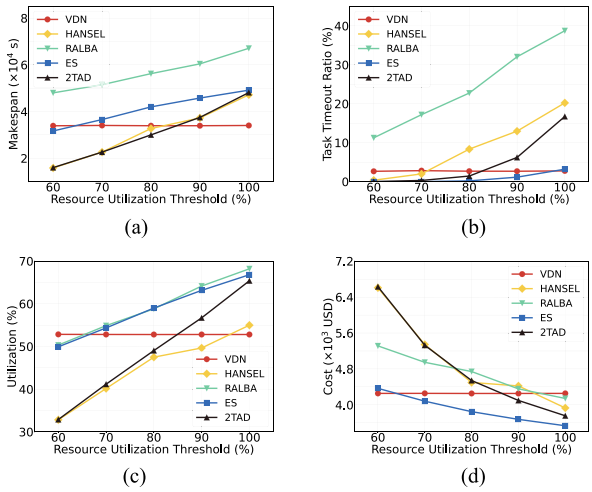


Fig. 11. Performance under various resource utilization thresholds. (a) Makespan. (b) Task timeout ratio. (c) Utilization. (d) Cost.

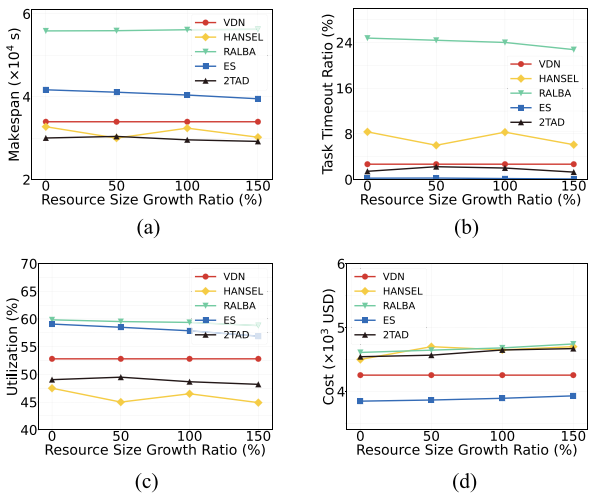


Fig. 12. Performance under different scaling resource size. (a) Makespan. (b) Task timeout ratio. (c) Utilization. (d) Cost.

2TAD is basically consistent, where the makespan, task timeout ratio, utilization, and cost and are growing with the increase of the task data size. The growth of the data volume of a single task may increase the task execution time as well as the task makespan. In this situation, the tasks are more likely to be over the time limit, resulting in an increasing task timeout ratio. Moreover, increasing the data size inevitably leads to an increase of resource consumption for both data transmission and task computing, and we have to pay more cost. By contrast, *RALBA* and *ES* maintain a stable resource utilization by renting more resources as the increase of data size. Therefore, the cost of these two algorithms also grows the fastest.

We conduct an evaluation to verify the performance under different resource utilization constraints. We increase the resource utilization threshold from 60% to 100% with a step of 10%, while keeping other experimental parameters unchanged. The experiment results are shown in Fig. 11. We find that as the resource utilization threshold grows, the makespan, resource utilization and task timeout ratio of *HANSEL*, *2TAD*, *RALBA* and *ES* are all increasing, but the total cost is decreasing. By contrast, the performance of *VDN* remains stable under different resource utilization constraints. Since *VDN* only performs task scheduling, its performance is unrelated to resource utilization. Whereas, the other four algorithms may rent fewer computing resources to reduce the rental cost, which leads to an increase of resource utilization. However, renting fewer computing resources also increases the likelihood that the time-varying workload exceeds the system's computing capacity, resulting in an increasing proportion of task timeout.

In order to compare the impact of different resource instances on algorithm performance, we gradually increase the resource size for scaling. Here we set the average intensity $\bar{\lambda}$ to 3/s. The experimental results are shown in the Fig. 12. Since *VDN* only considers task scheduling, the performance of *VDN* is not affected at all. Whereas, *HANSEL* only considers resource scaling, the increase of resource adjustment amplitude may result in a large fluctuation of performance. In contrast, the change of resource size has a small impact on *RALBA*, *ES* and *2TAD*. They jointly consider task scheduling and resource scaling, which can effectively schedule tasks from high-load data centers to the low-load ones, thereby mitigating the fluctuation of performance. We can also find that as the resource size increases, the total cost of these three algorithms gradually increases, while the other three metrics gradually decrease.

TABLE V
THE OVERHEAD OF DIFFERENT STATE REPRESENTATION MODELS

State Representation Models	MLP	GRU	DGCN
Communication Overhead	60 Bytes	60 Bytes	60 Bytes
Avg. GPU Utilization	5%	24%	11%
Avg. GPU Memory Usage	1231 MB	1275 MB	1237 MB
Time	0.25 ms	0.82 ms	1.07 ms

Finally, we compare the overhead and the performance of different state representation models include MLP, GRU and DGCN. In the experiments, we change the state representation model while retaining the remaining modules of *2TAD*. We uniformly set the output dimension of different state representation models to 15, that is, 60 bytes. The experimental results are shown in Table V and Fig. 13. We find that the running times for the three models are all on the order of milliseconds. The DGCN model takes much time to perform convolution calculations. By contrast, the GRU model reduces time by consuming more GPU resources. In addition, the three models have similar average GPU memory usage. Using different state representation models, *2TAD* achieves similar resource utilization and total cost. However, the DGCN-based *2TAD* has the lowest makespan and task timeout ratio, followed by the GRU-based *2TAD* and the MLP-based *2TAD*. Because the DGCN model can extract the graph structured features between data centers while the GRU model can only depict the sequence structured features and the MLP model does not consider the intrinsic structure.

All the above comparative simulation results indicate that by jointly optimizing task scheduling and resource scaling, *2TAD*

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:09:31 UTC from IEEE Xplore. Restrictions apply.

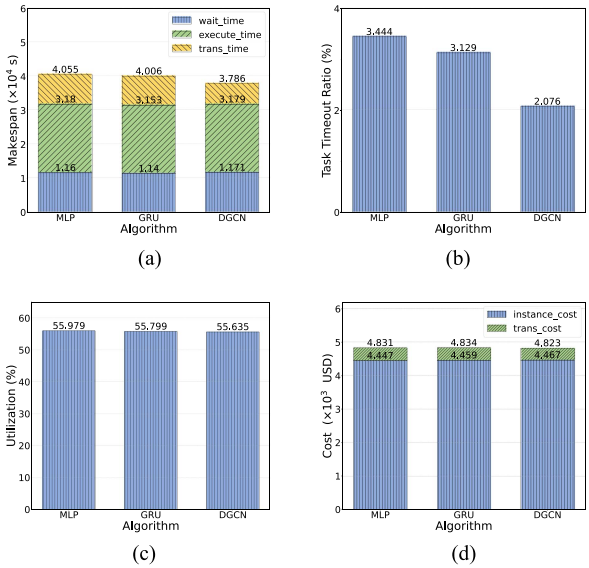


Fig. 13. Performance of different state representation models. (a) Makespan. (b) Task timeout ratio. (c) Utilization. (d) Cost.

can effectively improve the quality of service by reducing the makespan and the timeout ratio of tasks while maintaining a reasonable cost.

VII. Conclusion and Future Work

In this paper, we studied the joint optimization problem of task scheduling and resource scaling in the multi-data center system. Since the task scheduling is usually performed in a short timescale and the resources scaling is usually performed in a long timescale, we proposed a two-timescale multi-agent deep reinforcement learning algorithm to address this problem. To characterize the spatial dynamic of workloads in the multi-data center system, we proposed a DGCN based state representation model. The evaluation results show that the proposed approach can reduce the makespan and the timeout ratio while maintaining a reasonable cost. Our future work may focus on the service migration and the fault-tolerant task scheduling in an unreliable system, which are rarely considered in the literature.

References

[1] T. Shi, H. Ma, G. Chen, and S. Hartmann, "Location-aware and budget-constrained service deployment for composite applications in multi-cloud environment," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 8, pp. 1954–1969, Aug. 2020.

[2] D. C. N. De Azevedo and C. A. G. Ferraz, "A survey on autonomic multi-cloud computing," in *Proc. IEEE Int. Conf. Cloud Comput. Technol. Sci.*, 2022, pp. 33–40.

[3] D. McCarthy et al., "IDC FutureScape: Worldwide cloud 2023 predictions," International Data Corporation, Ltd., Oct. 2022. [Online]. Available: https://www.idc.com/getdoc.jsp?containerId=US48602322

[4] J. Wang, G. Zhao, H. Xu, Y. Zhao, X. Yang, and H. Huang, "TRUST: Real-time request updating with elastic resource provisioning in clouds," in *Proc. IEEE Conf. Comput. Commun.*, 2022, pp. 620–629.

[5] A. Beloglazov and R. Buyya, "Managing overloaded hosts for dynamic consolidation of virtual machines in cloud data centers under quality of service constraints," *IEEE Trans. Parallel Distrib. Syst.*, vol. 24, no. 7, pp. 1366–1379, Jul. 2013.

[6] D. Yi, X. Zhou, Y. Wen, and R. Tan, "Efficient compute-intensive job allocation in data centers via deep reinforcement learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 6, pp. 1474–1485, Jun. 2020.

[7] A. Marahatta, S. Pirbhulal, F. Zhang, R. M. Parizi, K.-K. R. Choo, and Z. Liu, "Classification-based and energy-efficient dynamic task scheduling scheme for virtualized cloud data center," *IEEE Trans. Cloud Comput.*, vol. 9, no. 4, pp. 1376–1390, Fourth Quarter, 2021.

[8] L. Ye, Y. Xia, S. Tao, C. Yan, R. Gao, and Y. Zhan, "Reliability-aware and energy-efficient workflow scheduling in IaaS clouds," *IEEE Trans. Autom. Sci. Eng.*, vol. 20, no. 3, pp. 2156–2169, Jul. 2023.

[9] Q.-H. Zhu, H. Tang, J.-J. Huang, and Y. Hou, "Task scheduling for multi-cloud computing subject to security and reliability constraints," *IEEE/CAA J. Automatica Sinica*, vol. 8, no. 4, pp. 848–865, Apr. 2021.

[10] X. Tang, "Reliability-aware cost-efficient scientific workflows scheduling strategy on multi-cloud systems," *IEEE Trans. Cloud Comput.*, vol. 10, no. 4, pp. 2909–2919, Fourth Quarter, 2022.

[11] S. Long, W. Long, Z. Li, K. Li, Y. Xia, and Z. Tang, "A game-based approach for cost-aware task assignment with QoS constraint in collaborative edge and cloud environments," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 7, pp. 1629–1640, Jul. 2021.

[12] Y. Zhang, B. Di, Z. Zheng, J. Lin, and L. Song, "Distributed multi-cloud multi-access edge computing by multi-agent reinforcement learning," *IEEE Trans. Wireless Commun.*, vol. 20, no. 4, pp. 2565–2578, Apr. 2021.

[13] A. Suzuki, M. Kobayashi, and E. Oki, "Multi-agent deep reinforcement learning for cooperative computing offloading and route optimization in multi cloud-edge networks," *IEEE Trans. Netw. Serv. Manag.*, vol. 20, no. 4, pp. 4416–4434, Dec. 2023.

[14] H. Ke, H. Wang, W. Sun, and H. Sun, "Adaptive computation offloading policy for multi-access edge computing in heterogeneous wireless networks," *IEEE Trans. Netw. Serv. Manag.*, vol. 19, no. 1, pp. 289–305, Mar. 2022.

[15] K. Oh, M. Zhang, A. Chandra, and J. Weissman, "Network cost-aware geo-distributed data analytics system," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 6, pp. 1407–1420, Jun. 2022.

[16] L. Osmani, T. Kauppinen, M. Komu, and S. Tarkoma, "Multi-cloud connectivity for kubernetes in 5G networks," *IEEE Commun. Mag.*, vol. 59, no. 10, pp. 42–47, Oct. 2021.

[17] S. N. Srirama, M. Adhikari, and S. Paul, "Application deployment using containers with auto-scaling for microservices in cloud environment," *J. Netw. Comput. Appl.*, vol. 160, 2020, Art. no. 102629.

[18] A. Gambi, M. Pezzè, and G. Toffetti, "Kriging-based self-adaptive cloud controllers," *IEEE Trans. Serv. Comput.*, vol. 9, no. 3, pp. 368–381, May/Jun. 2016.

[19] M. Xu et al., "CoScal: Multi-faceted scaling of microservices with reinforcement learning," *IEEE Trans. Netw. Serv. Manag.*, vol. 19, no. 4, pp. 3995–4009, Dec. 2022.

[20] S. Kardani-Moghaddam, R. Buyya, and K. Ramamohanarao, "ADRL: A hybrid anomaly-aware deep reinforcement learning-based resource scaling in clouds," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 3, pp. 514–526, Mar. 2021.

[21] M. Yan et al., "HANSEL: Adaptive horizontal scaling of microservices using Bi-LSTM," *Appl. Soft Comput.*, vol. 105, 2021, Art. no. 107216.

[22] Y. Al-Dhuraibi, F. Paraiso, N. Djarallah, and P. Merle, "Elasticity in cloud computing: State of the art and research challenges," *IEEE Trans. Serv. Comput.*, vol. 11, no. 2, pp. 430–447, Mar./Apr. 2018.

[23] A. Kwan, J. Wong, H.-A. Jacobsen, and V. Muthusamy, "HyScale: Hybrid and network scaling of dockerized microservices in cloud data centres," in *Proc. IEEE 39th Int. Conf. Distrib. Comput. Syst.*, 2019, pp. 80–90.

[24] Z. Chen, K. Lin, B. Lin, X. Chen, X. Zheng, and C. Rong, "Adaptive resource allocation and consolidation for scientific workflow scheduling in multi-cloud environments," *IEEE Access*, vol. 8, pp. 190173–190183, 2020.

[25] A. B. Kanbar and K. Faraj, "Region aware dynamic task scheduling and resource virtualization for load balancing in IoT–fog multi-cloud environment," *Future Gener. Comput. Syst.*, vol. 137, pp. 70–86, 2022.

[26] F. Rossi, V. Cardellini, F. L. Presti, and M. Nardelli, "Dynamic multi-metric thresholds for scaling applications using reinforcement learning," *IEEE Trans. Cloud Comput.*, vol. 11, no. 2, pp. 1807–1821, Second Quarter, 2023.

[27] W. Iqbal, A. Erradi, M. Abdullah, and A. Mahmood, "Predictive auto-scaling of multi-tier applications using performance varying cloud resources," *IEEE Trans. Cloud Comput.*, vol. 10, no. 1, pp. 595–607, First Quarter, 2022.

[28] M. Abdullah, W. Iqbal, J. L. Berral, J. Polo, and D. Carrera, "Burst-aware predictive autoscaling for containerized microservices," *IEEE Trans. Serv. Comput.*, vol. 15, no. 3, pp. 1448–1460, May/Jun. 2022.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:09:31 UTC from IEEE Xplore. Restrictions apply.

[29] T. Shi, H. Ma, G. Chen, and S. Hartmann, "Cost-effective web application replication and deployment in multi-cloud environment," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 8, pp. 1982–1995, Aug. 2022.
[30] T. Yang, Y. Hou, S. Cai, J. Yu, and H. Pen, "Multi-data center tie-line power smoothing method based on demand response," *IEEE Trans. Cloud Comput.*, early access, Jun. 6, 2024, doi: 10.1109/TCC.2024.3410377.
[31] C. Mouradian, S. Kianpisheh, M. Abu-Lebdeh, F. Ebrahimnezhad, N. T. Jahromi, and R. H. Glitho, "Application component placement in NFV-based hybrid cloud/fog systems with mobile fog nodes," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 5, pp. 1130–1143, May 2019.
[32] J. Li et al., "Maximizing user service satisfaction for delay-sensitive IoT applications in edge computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 5, pp. 1199–1212, May 2022.
[33] Z. Yan, J. Ge, Y. Wu, L. Li, and T. Li, "Automatic virtual network embedding: A deep reinforcement learning approach with graph convolutional networks," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 6, pp. 1040–1057, Jun. 2020.
[34] Network latency between data centers in amazon cloud platform, 2023. [Online]. Available: https://www.cloudping.co/grid
[35] J. Tang, M. M. Jalalzai, C. Feng, Z. Xiong, and Y. Zhang, "Latency-aware task scheduling in software-defined edge and cloud computing with erasure-coded storage systems," *IEEE Trans. Cloud Comput.*, vol. 11, no. 2, pp. 1575–1590, Second Quarter, 2023.
[36] N. Cheng et al., "Space/aerial-assisted computing offloading for IoT applications: A learning-based approach," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 5, pp. 1117–1129, May 2019.
[37] S. Nabi, M. Ibrahim, and J. M. Jimenez, "DRALBA: Dynamic and resource aware load balanced scheduling approach for cloud computing," *IEEE Access*, vol. 9, pp. 61283–61297, 2021.
[38] J. Choi et al., "VM auto-scaling methods for high throughput computing on hybrid infrastructure," *Cluster Comput.*, vol. 18, pp. 1063–1073, 2015.



Shuangwu Chen (Member, IEEE) received the BS and PhD degrees from the University of Science and Technology of China (USTC), Hefei, China, in 2011 and 2016, respectively. He is currently an associate professor with the USTC, Hefei, China, and an associate research fellow with the Institute of Artificial Intelligence, Hefei Comprehensive National Science Center, Hefei, China. His research interests include future network, multimedia communication, and stochastic optimization.



Jiangming Li received the BS degree from the University of Science and Technology of China (USTC) in 2019. He is currently working toward the PhD degree with the School of Information Science and Technology, USTC. His recent research interests include cloud computing, network security, and network optimization.



Qifeng Yuan received the BS degree in 2017 from the University of Science and Technology of China (USTC), Hefei, China, where he is currently working toward the PhD degree with the School of Information Science and Technology. His research interests include future network, mobile edge computing, and network resource optimization.



Huasen He (Member, IEEE) received the BS degree in automation from the University of Science and Technology of China, Hefei, China, in 2013, the MS degree in signal processing and communications, and the PhD degree in digital communications from the University of Edinburgh, Edinburgh, U.K., in 2014 and 2018, respectively. He is currently an associate research fellow with the School of Information Science and Technology, USTC. His research interests include stochastic geometry, future networks, and signal processing for communication.



Sen Li received the master of engineering degree in computer technology in 2011. He is currently working with the Information Center of Guangxi. His research interests include e-government networks and next generation network.



Jian Yang (Senior Member, IEEE) received the BS and PhD degrees from the University of Science and Technology of China (USTC), Hefei, China, in 2001 and 2006, respectively. From 2006 to 2008, he was a postdoctoral scholar with the Department of Electronic Engineering and Information Science, USTC. Since 2008, he has been employed as an associate professor with the Department of Automation, USTC. He is currently a professor with the School of Information Science and Technology, USTC. His research interests include future network, distributed system design, modeling & optimization, multimedia over wired & wireless, and stochastic optimization.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:09:31 UTC from IEEE Xplore. Restrictions apply.